Assignment 1 - Benchmarking Cryptography

Group: PL4G3

- *Filipe Huang*: up202406540
- *Diego Nóbrega*: up202407575

Introduction and Goal

The objective of this assignment is to evaluate the performance of three core cryptographic mechanisms — **AES(CTR mode)**, **RSA**, and **SHA-256** — when processing files of increasing sizes ***from 8 bytes to 2097152 bytes(=2MiB)***.

The goal is to measure **execution time** (in *microseconds*) and **throughput** (*bytes/microseconds*), analyse *scaling behaviour*, and understand the practical trade-offs of **symmetric**, **asymmetric**, and **hashing** primitives.

Experimental Setup

The environment utilized Python 3.12 and the cryptography library version 46.0.6, as detailed in the project's README. Specifically, the benchmarks were run on two diferent sistems:

- an AMD Ryzen 7 5825U with Radeon Graphics and 16GB of RAM, running the virtual machine OracleVirtualBox, with Fedora (64-bits) operating system.

- Intel® Core™ i9-14900 with 32GB of RAM, using the WSL(Windows Subsystem for Linux)

Execution times were measured using the **timeit** module, with *multiple repetitions* and *statistical averaging* to ensure reliable results.

Only the time of the cryptographic operations was measured, excluding file generation and other overhead.

File Generation

The extract of code below generates **cryptographically secure random** test files(binary) *from 8 bytes to 2 MiB*.

The files created here serve as input for all benchmarks.

```
def gerar_ficheiros():
    for tamanho in tamanhos:
        nome_ficheiro = f"random_{tamanho}.txt"
        with open(nome_ficheiro, "wb") as f:
```

```
        f.write(os.urandom(tamanho))
```

Timing Methodology

This component measures the *execution time* of **cryptographic operations**, including *symmetric encryption*, *asymmetric encryption*, and *hashing*.

It will examine *how the execution time changes* as the *input size increases*, providing insight into the efficiency of each method(to ensure reliable results, measurements are repeated and averaged to reduce variability).

The results enable a comparison between different approaches and are visualized through graphs(will be shown below while comparing) for easier interpretation.

```
def tempo_execucao(funcao, dados, repeticoes=100):
    funcao(dados)  # Warm-up
    tempos = timeit.repeat(lambda: funcao(dados), number=1,
         repeat=repeticoes)
    tempos_us = [t * 1_000_000 for t in tempos]
    return statistics.mean(tempos_us), statistics.stdev(tempos_us)
```

AES Implementation (CTR Mode)

AES encryption and decryption were implemented using CTR mode with a 256-bit key.

```
def AES_cifra(dados):
    cifra = Cipher(algorithms.AES(chave), modes.CTR(nonce))
    encriptador = cifra.encryptor()
    return encriptador.update(dados) + encriptador.finalize()

def AES_decifra(dados):
    cifra = Cipher(algorithms.AES(chave), modes.CTR(nonce))
    decriptador = cifra.decryptor()
    return decriptador.update(dados) + decriptador.finalize()
```

CTR mode transforms AES from a *block cipher* into a *stream cipher*. This eliminates the need for padding and enables continuous data processing, which improves performance.

RSA Implementation(hybrid)

This is a *hybrid implementation*, since RSA cannot efficently encrypt large data.

```
def RSA_cifra_custom(m):
    r = os.urandom(32)                      # 256-bit random key
    c0 = rsa_funcao(r)                      # RSA-encrypt r
    blocos = [c0]
    for i in range(math.ceil(len(m)/32)):
        h = hashlib.sha256(i.to_bytes(4,'big') + r).digest()
        blocos.append(xor_bytes(h[:len(m_i)], m_i))
    return b''.join(blocos)
```

Note: this is a simplified scheme, not a semantically secure padding, but still valid for performance benchmarking

```
def RSA_decifra_custom(ct):
    c0 = ct[:256]
    r = rsa_inverso(c0)
    ct_restante = ct[256:]
    blocos_texto = []
    for i in range(math.ceil(len(ct_restante) / 32)):
```

```
        c_i = ct_restante[i*32 : (i+1)*32]
        h = hashlib.sha256(i.to_bytes(4, 'big') + r).digest()
        blocos_texto.append(xor_bytes(h[:len(c_i)], c_i))
    return b''.join(blocos_texto)
```

The corresponding decryption function extracts the original seed r by applying the RSA inverse operation (using the private key) on the first 256 bytes. It then reverses the XOR operations on the remaining blocks to recover the plaintext

SHA-256 Hashing

SHA-256 is used to compute hash values.

This operation serves as a reference for comparing encryption performance.

```
def gerador_SHA256(dados):
digest = hashlib.sha256()
digest.update(dados) return
digest.digest()
```

Results

(Measured with the second system)

Execution time(in microsenconds, average over repetition)

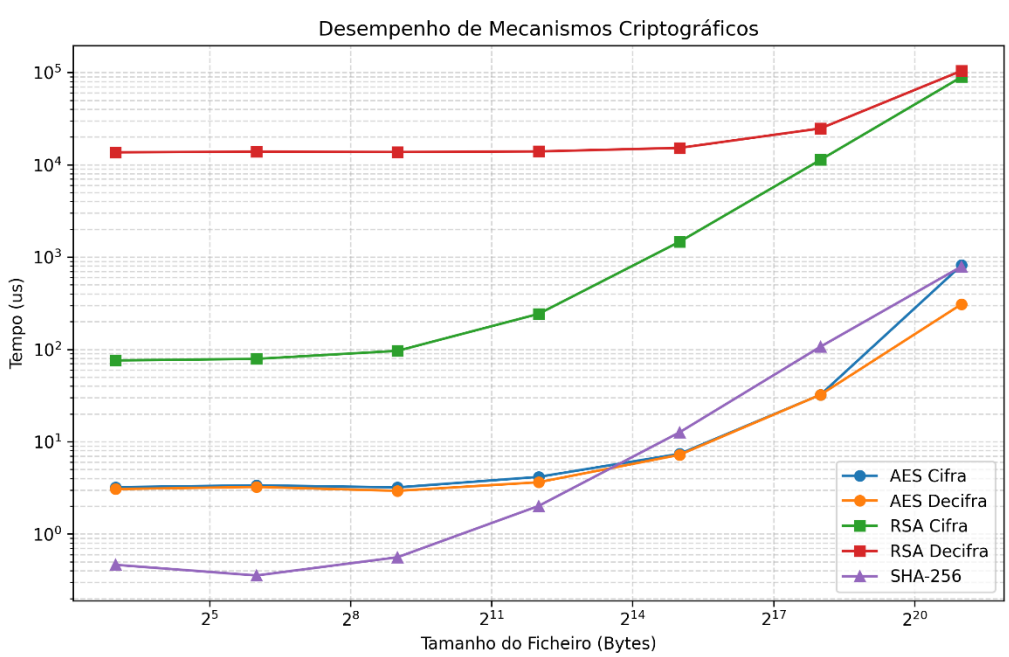
Size(bytes)	AES(Enc)	AES(Dec)	RSA(Enc)	RSA(Dec)	SHA-256
8	3.21	3.07	75.91	13635.93	0.46
64	3.37	3.23	78.92	13871.86	0.36
512	3.20	2.94	96.43	13748.67	0.56
4096	4.15	3.64	242.17	13943.17	2.01
32768	7.38	7.21	1465.54	15239.39	12.63
262144	32.28	32.31	11321.89	24769.36	107.03
2097152	816.50	309.21	89452.11	104327.53	791.72

Note: the results are presented in microsenconds for clearer readability, as values in seconds would contain many decimal places.

log-log plot

As the size of the files and execution time varies in a *considerably large scale*, it would be difficult to observe values in a **linear scale**.

So we opted to use ***log-log plot*** to keep all the data visible and easier to compare.



Performance Analysis

Execution Variability and Cold Start

When running a fixed cryptographic algorithm over the same file multiple times, the results change significantly between the first execution and the subsequent ones. The first run is typically much slower due to a "cold start" effect—the operating system must load the cryptographic libraries into memory and populate the CPU cache. Subsequent runs become significantly faster and more stable. To mitigate this and ensure statistical significance, a warm-up execution was included in our tempo_execucao function.

Furthermore, if an algorithm is run over multiple randomly generated files of a fixed size, the execution time remains highly consistent (assuming the system is warmed up). Cryptographic primitives like AES and SHA process data logically and linearly; therefore, the specific random content of the bytes does not impact the processing time, only the total file size does.

AES vs RSA Encryption

AES outperforms *RSA* across all file sizes by a wide margin. AES processes data as a stream cipher with minimal overhead, while RSA relies on modular exponentiation—a computationally expensive operation. For a 2 MiB file, AES encryption takes 816 µs compared to RSA's 89,452 µs.

AES processes data *linearly* with *constant-time block operations* (stream-cipher style). *RSA*, even in hybrid form, still performs one full public-key modular exponentiation on the 32-byte random key r plus 32-byte SHA-256 operations for every block. The cost of the RSA public-key operation dominates and grows with file size because more blocks must be hashed and XORed.

AES vs SHA-256

AES and *SHA-256* show similar scaling behavior, with *SHA-256* being *slightly faster*. For small files (8 bytes), SHA-256 takes 0.46 µs versus AES's 3.21 µs. At 2 MiB, the difference narrows to 791 µs versus 816 µs. This is expected since hashing is a one-way operation, while encryption must produce reversible output.

Both algorithms are *highly efficient* and scale proportionally to the input size. This confirms that, when only integrity (not confidentiality) is needed, SHA-256 is the lightest option; when confidentiality is required, *AES-CTR* adds *almost no extra cost*.

RSA Encryption vs Decryption

RSA **decryption** is *consistently slower* than **encryption**. At 8 bytes, encryption takes *75.91 µs* while decryption takes *13,635 µs* — a difference of two orders of magnitude. This gap persists across all sizes and reflects the use of a larger private exponent (d) compared to the public exponent (e=65537), making decryption computationally heavier.

Conclusion

The benchmarks confirm theoretical expectations:

- **AES-CTR** and **SHA-256** are *fast*, linear-scaling primitives ideal for *large volumes of data*.
- **RSA** (*even in hybrid form*) is the bottleneck and should be reserved for key exchange or small payloads.

These results explain why real-world systems always combine RSA (for key agreement) with AES (for the actual data).

Bibliography

- Cryptography library documentation
- Python **timeit** and **statistics** modules
- Course materials

AI Usage

AI tools were used only to improve the structure and clarity of the report.

All implementations and experimental work were developed independently.